Directory Content Report

File: install.bat

```
@echo off

REM Windows batch script to set up the Source Code to PDF Generator environment

echo Source Code to PDF Generator - Windows Setup

echo ========================================

REM Check if Python is available

python --version &gt;nul 2&gt;&amp;1

if errorlevel 1 (

    echo Error: Python is not installed or not in PATH

    echo Please install Python 3.6 or higher and ensure it's in your PATH

    pause

    exit /b 1

)

REM Check Python version

for /f "tokens=2" %%i in ('python --version 2^&gt;^&amp;1') do set PYTHON_VERSION=%%i

for /f "tokens=1,2,3 delims=." %%a in ("%PYTHON_VERSION%") do (

    set PYTHON_MAJOR=%%a

    set PYTHON_MINOR=%%b

)

if %PYTHON_MAJOR% lss 3 (

    echo Error: Python 3.6 or higher is required. Current version: %PYTHON_VERSION%

    pause

    exit /b 1

)

if %PYTHON_MAJOR% equ 3 if %PYTHON_MINOR% lss 6 (

    echo Error: Python 3.6 or higher is required. Current version: %PYTHON_VERSION%
```

```
    pause
    exit /b 1
)

echo ✓ Python version %PYTHON_VERSION% is compatible

REM Create virtual environment
echo.
echo Creating virtual environment...
if exist "venv" (
    echo Virtual environment already exists. Removing...
    rmdir /s /q "venv"
)

python -m venv venv
if errorlevel 1 (
    echo Error: Failed to create virtual environment
    pause
    exit /b 1
)

echo ✓ Virtual environment created successfully

REM Upgrade pip in the virtual environment
echo.
echo Upgrading pip...
venv\Scripts\python.exe -m pip install --upgrade pip
if errorlevel 1 (
    echo Warning: Failed to upgrade pip, continuing anyway
)

REM Install requirements
echo.
echo Installing required packages...
```

```
if exist "requirements.txt" (
    venv\Scripts\python.exe -m pip install -r requirements.txt
) else (
    echo requirements.txt not found, installing default packages...
    venv\Scripts\python.exe -m pip install reportlab chardet
)

if errorlevel 1 (
    echo Error: Failed to install required packages
    pause
    exit /b 1
)

echo ✓ Required packages installed successfully

REM Verify installation
echo.
echo Verifying installation...
venv\Scripts\python.exe -c "import reportlab; import chardet; import tkinter; print('✓ All required packages
if errorlevel 1 (
    echo Error: Verification failed - some packages are not properly installed
    pause
    exit /b 1
)

echo.
echo Setup completed successfully!
echo.
echo To use the Source Code to PDF Generator:
echo 1. Run this command to activate the virtual environment:
echo    venv\Scripts\activate
echo.
echo 2. Then run the application:
```

```
echo    python repo_to_saip.py

echo.

echo You can also run without activating by using the full path:

echo    venv\Scripts\python.exe repo_to_saip.py

echo.

pause
```

File: README.md

```
# Source Code to PDF Generator for Intellectual Property Rights Applications

## Problem Statement

When applying for intellectual property rights as Mohammed Ismail, I encountered issues with the "authored t

The challenge was to create a comprehensive, organized, and properly formatted PDF document that contains al

## Solution Overview

The `repo_to_saip.py` script automatically generates a comprehensive PDF report containing all source code f

## Key Features

### 1. Intelligent File Filtering
- **GitIgnore Support**: Recursively traverses directories while honoring `.gitignore` patterns at all level
- **Binary File Detection**: Automatically identifies and excludes binary files to focus only on text/source
- **Smart Exclusions**: Respects common project exclusions like `node_modules`, build directories, logs, and

### 2. Multi-Format Archive Support
- **Archive Formats**: Supports multiple archive formats including ZIP, TAR, TAR.GZ, TGZ, and TAR.BZ2 as inp
- **Flexible Input**: Can process both live directories and archived codebases
- **Automatic Extraction**: Seamlessly handles archive extraction and processing

### 3. Encoding and Content Handling
- **Automatic Encoding Detection**: Detects and handles different file encodings automatically using chardet
- **Robust Reading**: Handles files with various character encodings while preserving content integrity
```

- **Error Resilience**: Continues processing even when individual files cannot be read

4. Professional PDF Output

- **Formatted Output**: Creates a well-formatted PDF with clear file headers and preserved code formatting
- **Organized Structure**: Each file is clearly labeled with its path for easy navigation
- **Content Management**: Manages large files by truncating extremely long content to maintain PDF usability

5. User Interface Options

- **Dual Interface**: Provides both GUI and command-line interfaces for flexibility
- **Progress Tracking**: Includes comprehensive progress tracking and status updates
- **Error Handling**: Comprehensive error handling with user-friendly messages

Installation Requirements

Prerequisites

- Python 3.6 or higher
- ReportLab library for PDF generation
- Chardet library for encoding detection

Installation

```bash
pip install reportlab chardet
```

Usage Instructions

Command-Line Interface

Basic Usage

```bash
python repo_to_saip.py &lt;input_path&gt; [output_pdf]
```

Examples

```bash
```

```
# Process a directory and create default output (directory_report.pdf)
python repo_to_saip.py /path/to/your/project

# Process a directory with custom output filename
python repo_to_saip.py /path/to/your/project my_source_code.pdf

# Process an archive file
python repo_to_saip.py /path/to/your/archive.zip source_code.pdf
```

Parameters

- `<input_path>`: Path to the directory or archive file to process (required)
- `[output_pdf]`: Path for the output PDF file (optional, defaults to "directory_report.pdf")

Graphical User Interface

Starting the GUI

```bash
python repo_to_saip.py
```

Running the script without arguments will launch the graphical user interface.

GUI Features

- **Input Selection**: Browse and select either a directory or an archive file
- **Output Configuration**: Specify the output PDF file location and name
- **Progress Visualization**: Real-time progress bar showing processing status
- **Status Updates**: Detailed status messages during processing
- **Log Display**: Comprehensive logging of all processing activities
- **Error Handling**: User-friendly error messages and validation

How This Tool Solves IP Rights Documentation Requirements

Complete Source Code Compilation

The script addresses the core requirement of IP rights applications by creating a complete compilation of al

Professional Presentation

- **Clear Organization**: Each file is clearly labeled with its full path for easy identification
- **Consistent Formatting**: Maintains code formatting while ensuring readability in PDF format
- **Professional Layout**: Uses ReportLab's professional document generation capabilities

Compliance with IP Standards

- **Comprehensive Coverage**: Includes all relevant source code while excluding non-essential files
- **File Integrity**: Preserves the original content and structure of source files
- **Traceability**: Maintains file paths and names for easy verification and cross-referencing

Efficiency and Reliability

- **Automated Processing**: Eliminates manual copying and pasting of source code
- **Consistent Results**: Produces the same quality output regardless of project size
- **Error Handling**: Manages exceptions gracefully to ensure reliable output

Technical Architecture

Core Components

1. **GitIgnoreProcessor**: Handles .gitignore pattern matching and file filtering
2. **Archive Extraction**: Supports multiple archive formats for flexible input
3. **Encoding Detection**: Uses chardet library for automatic encoding detection
4. **PDF Generation**: Leverages ReportLab for professional PDF creation
5. **GUI Framework**: Tkinter-based interface for user-friendly operation

Processing Workflow

1. **Input Validation**: Validates the input path (directory or archive)
2. **Archive Extraction**: If input is an archive, extracts to temporary directory
3. **File Discovery**: Recursively discovers files while respecting .gitignore rules
4. **Content Processing**: Reads and processes file content with encoding detection
5. **PDF Creation**: Generates the final PDF report with organized content

Benefits for IP Rights Applications

Time Savings

- **Automation**: Eliminates hours of manual source code compilation
- **One-Click Processing**: Simple operation for complex multi-file projects
- **Batch Processing**: Handles entire project repositories in one operation

Quality Assurance
- **Completeness**: Ensures all relevant source code is included
- **Consistency**: Maintains consistent formatting across all files
- **Accuracy**: Preserves original content without manual transcription errors

Professional Standards
- **Readable Output**: Optimized for review by IP examiners and legal professionals
- **Organized Structure**: Logical file organization for easy navigation
- **Comprehensive Coverage**: Full project documentation in standard format

Troubleshooting

Common Issues
- **Missing Dependencies**: Ensure ReportLab and chardet libraries are installed
- **Permission Errors**: Verify read/write permissions for input/output directories
- **Large Projects**: For very large projects, consider increasing system memory or processing in smaller ch

Error Messages
- **"ReportLab library is not installed"**: Install ReportLab using `pip install reportlab`
- **"Path does not exist"**: Verify the input path is correct and accessible
- **"No write permission"**: Check permissions for the output directory

Use Cases

Intellectual Property Applications
- **Patent Applications**: Documentation for software-related patents
- **Copyright Registration**: Source code compilation for copyright claims
- **Trademark Submissions**: Software code evidence for trademark applications
- **Licensing Documentation**: Complete source code for licensing agreements

Other Applications

- **Code Audits**: Comprehensive source code review documentation
- **Project Handoffs**: Complete project documentation for team transitions
- **Backup Documentation**: Source code preservation in PDF format
- **Legal Discovery**: Organized source code for legal proceedings

Best Practices

For IP Rights Applications

1. **Include All Relevant Code**: Ensure the directory contains all source files related to the IP claim
2. **Verify GitIgnore Settings**: Review .gitignore files to ensure no essential code is excluded
3. **Test Output**: Review the generated PDF to ensure completeness and readability
4. **Maintain Originals**: Keep original source files as backup to the PDF documentation

File Organization Tips

- Use descriptive filenames for the output PDF
- Organize source code in a clean directory structure before processing
- Update .gitignore files to properly exclude non-essential files
- Consider creating a dedicated branch/release for IP documentation

License and Usage Rights

This tool is provided to help developers and IP professionals meet documentation requirements for intellectu

File: repo_to_saip.py

```
#!/usr/bin/env python3
"""
Script to traverse a directory, respect .gitignore files, and compile all non-ignored
text file contents into a single PDF document.
"""

import os
import re
import sys
```

```
import tempfile
import zipfile
import tarfile
from pathlib import Path
import chardet
from typing import List, Set, Optional
import fnmatch
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext

# Try to import PDF generation libraries
try:
    from reportlab.pdfgen import canvas
    from reportlab.lib.pagesizes import letter, A4
    from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Preformatted
    from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
    from reportlab.lib.units import inch
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont
    PDF_AVAILABLE = True
except ImportError:
    PDF_AVAILABLE = False

class GitIgnoreProcessor:
    """Processes .gitignore files to determine which files should be ignored."""

    def __init__(self):
        self.ignores = []
        self.includes = []

    def add_pattern(self, pattern: str, is_negative: bool = False):
        """Add a pattern to the ignore list."""
        if pattern.startswith('!'):
```

```
            # This is a negative pattern (include)
            pattern = pattern[1:]
            self.includes.append(self._compile_pattern(pattern))
        else:
            self.ignores.append(self._compile_pattern(pattern))

    def _compile_pattern(self, pattern: str):
        """Compile a gitignore pattern to a regex."""
        # Normalize the pattern
        pattern = pattern.strip()

        # If it starts with /, it's anchored to the directory
        anchored = pattern.startswith('/')
        if anchored:
            pattern = pattern[1:]

        # If it ends with /, it only matches directories
        dir_only = pattern.endswith('/')
        if dir_only:
            pattern = pattern[:-1]

        # Convert gitignore pattern to regex
        regex_pattern = self._gitignore_to_regex(pattern)

        # Anchor to start if originally anchored
        if anchored:
            regex_pattern = '^' + regex_pattern
        else:
            # Allow matching at any level
            regex_pattern = '(/|^)' + regex_pattern

        return re.compile(regex_pattern), dir_only

    def _gitignore_to_regex(self, pattern: str) -&gt; str:
```

```
    """Convert a gitignore pattern to a regex pattern."""
    # Escape special regex characters
    pattern = re.escape(pattern)

    # Replace gitignore wildcards with regex equivalents
    pattern = pattern.replace('\\*', '.*')
    pattern = pattern.replace('\\?', '.')
    pattern = pattern.replace('\\[\\!', '[^')
    pattern = pattern.replace('\\]', ']')

    # Handle double asterisk (**) for directory recursion
    pattern = pattern.replace('.*.*', '.*')  # Simplified handling

    # Add end anchor if pattern doesn't contain wildcards that span directories
    if not ('.*' in pattern or '/' in pattern):
        pattern += '$'
    else:
        pattern += '(/|$)'

    return pattern

def is_ignored(self, file_path: str, is_dir: bool = False) -&gt; bool:
    """Check if a file or directory should be ignored based on patterns."""
    # Check include patterns first (negative patterns)
    for pattern, dir_only in self.includes:
        if dir_only and not is_dir:
            continue
        if pattern.search(file_path):
            return False  # This file is explicitly included

    # Check ignore patterns
    for pattern, dir_only in self.ignores:
        if dir_only and not is_dir:
            continue
```

```
            if pattern.search(file_path):
                return True  # This file is ignored

        return False


def extract_archive(archive_path: str, extract_to: str) -&gt; Optional[Path]:
    """Extract archive (ZIP, TAR, etc.) to temporary directory and return the path."""
    archive_path = Path(archive_path)
    extract_to = Path(extract_to)

    try:
        if not archive_path.exists():
            print(f"Archive file does not exist: {archive_path}")
            return None

        if archive_path.suffix.lower() == '.zip':
            with zipfile.ZipFile(str(archive_path), 'r') as zip_ref:
                zip_ref.extractall(str(extract_to))
        elif archive_path.suffix.lower() in ('.tar', '.tar.gz', '.tgz', '.tar.bz2'):
            with tarfile.open(str(archive_path), 'r:*') as tar_ref:
                tar_ref.extractall(str(extract_to))
        else:
            print(f"Unsupported archive format: {archive_path.suffix}")
            return None

        # Return the extracted directory
        return extract_to
    except zipfile.BadZipFile:
        print(f"Invalid ZIP file: {archive_path}")
        return None
    except tarfile.TarError as e:
        print(f"Invalid TAR file: {archive_path} - {e}")
        return None
```

```
    except PermissionError:
        print(f"Permission denied when extracting: {archive_path}")
        return None
    except Exception as e:
        print(f"Error extracting archive {archive_path}: {e}")
        return None


def load_gitignore_patterns(directory: Path) -&gt; GitIgnoreProcessor:
    """Load and process .gitignore files in a directory hierarchy."""
    processor = GitIgnoreProcessor()

    # Walk up the directory tree to get all parent .gitignore files
    current = directory
    while current != current.parent:
        gitignore_path = current / '.gitignore'
        if gitignore_path.exists():
            with open(str(gitignore_path), 'r', encoding='utf-8') as f:
                for line in f:
                    line = line.strip()
                    if line and not line.startswith('#') and not line.startswith('!'):
                        processor.add_pattern(line)
                    elif line.startswith('!'):
                        processor.add_pattern(line, is_negative=True)

        current = current.parent

    return processor

def is_binary_file(file_path: Path) -&gt; bool:
    """Check if a file is binary by reading the first few bytes."""
    try:
        with open(str(file_path), 'rb') as f:
            chunk = f.read(1024)  # Read first 1KB
```

```
            # Check for null bytes or other binary indicators
            if b'\x00' in chunk:
                return True

            # Try to decode as text - if it fails, it's likely binary
            try:
                chunk.decode('utf-8')
                return False
            except UnicodeDecodeError:
                return True
    except:
        return True

def detect_encoding(file_path: Path) -&gt; str:
    """Detect the encoding of a file."""
    with open(str(file_path), 'rb') as f:
        raw_data = f.read()
        result = chardet.detect(raw_data)
        encoding = result['encoding']
        confidence = result['confidence']

        # If confidence is low, default to utf-8
        if confidence &lt; 0.7:
            return 'utf-8'

        return encoding

def read_file_content(file_path: Path) -&gt; Optional[str]:
    """Read file content with proper encoding detection."""
    try:
        # First try UTF-8
        try:
            with open(str(file_path), 'r', encoding='utf-8') as f:
```

```
                return f.read()
        except UnicodeDecodeError:
            # If UTF-8 fails, detect encoding
            encoding = detect_encoding(file_path)
            with open(str(file_path), 'r', encoding=encoding) as f:
                return f.read()
    except Exception as e:
        print(f"Error reading file {file_path}: {e}")
        return None

def collect_files_with_gitignore(root_path: Path) -&gt; List[Path]:
    """Collect all non-ignored files in the directory tree."""
    all_files = []

    # Get gitignore processor for the root
    root_gitignore = load_gitignore_patterns(root_path)

    for current_dir, dirs, files in os.walk(root_path):
        current_path = Path(current_dir)

        # Load gitignore for this specific directory (for nested gitignores)
        dir_gitignore = load_gitignore_patterns(current_path)

        # Filter out directories that should be ignored
        dirs_to_remove = []
        for d in dirs:
            dir_path = current_path / d
            rel_path = dir_path.relative_to(root_path).as_posix()

            # Check if this directory should be ignored by any gitignore in the hierarchy
            if root_gitignore.is_ignored(rel_path, is_dir=True) or dir_gitignore.is_ignored(d, is_dir=True):
                dirs_to_remove.append(d)

        for d in dirs_to_remove:
```

```
            dirs.remove(d)

        # Process files in this directory
        for file in files:
            file_path = current_path / file
            rel_path = file_path.relative_to(root_path).as_posix()

            # Check if file should be ignored by any gitignore in the hierarchy
            if not root_gitignore.is_ignored(rel_path) and not dir_gitignore.is_ignored(file):
                all_files.append(file_path)

    return all_files

def create_pdf_report(files_content: List[tuple], output_path: str):
    """Create a PDF report with the collected file contents."""
    if not PDF_AVAILABLE:
        print("Error: ReportLab library is not installed.")
        print("Install it using: pip install reportlab")
        return False

    doc = SimpleDocTemplate(str(output_path), pagesize=A4)
    styles = getSampleStyleSheet()

    # Create a custom style for file content
    file_content_style = ParagraphStyle(
        'FileContent',
        parent=styles['Normal'],
        fontName='Courier',
        fontSize=8,
        leading=10,
        spaceAfter=12
    )

    stor
```

```
... [Content truncated for PDF]
```

File: setup_and_run.bat

```
@echo off

REM Simple Setup and Run Script for Source Code to PDF Generator

echo Source Code to PDF Generator - Quick Setup and Run

echo ================================================

REM Check if Python is available

python --version &gt;nul 2&gt;&amp;1

if errorlevel 1 (

    echo Error: Python is not installed or not in PATH

    pause

    exit /b 1

)

REM Install required packages if missing

python -c "import reportlab" &gt;nul 2&gt;&amp;1

if errorlevel 1 (

    echo Installing ReportLab...

    python -m pip install reportlab

)

python -c "import chardet" &gt;nul 2&gt;&amp;1

if errorlevel 1 (

    echo Installing chardet...

    python -m pip install chardet

)

REM Run the application

if [%1]==[] (

    REM No arguments provided, run GUI mode
```

```
    echo Starting Source Code to PDF Generator...
    python repo_to_saip.py
) else (
    REM Arguments provided, run command-line mode
    echo Starting Source Code to PDF Generator...
    python repo_to_saip.py %*
)

echo.
echo Application closed.
pause
```

File: Usecases\command_line_usage_screenshot_placeholder.png.txt

```
# Command-Line Usage Screenshot

This is a placeholder for the command-line usage screenshot showing:

- Terminal/command prompt execution

- Input and output parameters

- Progress indicators

- Success/failure messages
```

File: Usecases\gui_interface_screenshot_placeholder.png.txt

```
# GUI Interface Screenshot

This is a placeholder for the GUI interface screenshot showing:

- Input directory selection

- Output PDF specification

- Progress bar

- Log display

- Generate PDF button
```

File: Usecases\pdf_output_example_screenshot_placeholder.png.txt

```
# PDF Output Example Screenshot

This is a placeholder for the PDF output example screenshot showing:

- Generated PDF with organized source code

- File headers and proper formatting

- Multiple files in a single document

- Clean, professional layout
```

File: Usecases\README.md

```
# Use Cases for Source Code to PDF Generator


## Intellectual Property Rights Applications


This application is specifically designed to support intellectual property rights applications, particularly


### 1. Complete Source Code Compilation for IP Applications

- **Scenario**: When applying for intellectual property rights, you need to submit complete source code in P

- **Solution**: The application compiles all source code files from a directory into a single, organized PDF

- **Benefit**: Ensures all required code is included and properly formatted for IP office submission


### 2. GitIgnore-Aware Processing

- **Scenario**: You want to include only relevant source files while excluding build artifacts, logs, and te

- **Solution**: The application respects `.gitignore` files to exclude unnecessary files automatically

- **Benefit**: Creates a clean, professional document without irrelevant files


### 3. Multi-Format Archive Support

- **Scenario**: You have source code in archive format (ZIP, TAR, etc.) that needs to be converted to PDF

- **Solution**: The application can process archived codebases directly

- **Benefit**: No need to manually extract archives before processing


### 4. Encoding Detection and Handling

- **Scenario**: Source code files have different character encodings
```

- **Solution**: The application automatically detects and handles various file encodings
- **Benefit**: Ensures all source code is properly readable in the output PDF

5. GUI and Command-Line Interface
- **Scenario**: Different users prefer different interaction methods
- **Solution**: Offers both GUI and command-line interfaces
- **Benefit**: Flexible usage based on user preference and automation needs

Screenshots

GUI Interface
The main GUI interface allows users to:
- Select input directory or archive
- Specify output PDF location
- Monitor progress with visual indicators
- View processing logs

Command-Line Usage
For automated or advanced usage, the command-line interface allows:
- Direct processing with specified input/output paths
- Batch processing capabilities
- Integration into automated workflows

IP Rights Application Process Integration

Before Application Submission
1. Prepare your complete source code directory
2. Ensure `.gitignore` files properly exclude unnecessary files
3. Run the application to generate the PDF
4. Review the generated PDF for completeness

During Application Process
- Submit the generated PDF as part of your IP rights application
- The organized format helps examiners review your code

- Complete documentation demonstrates the scope of your intellectual property

Best Practices for IP Documentation

1. **Include All Relevant Code**: Ensure your source directory contains all code that forms part of your IP
2. **Proper Exclusions**: Use `.gitignore` to exclude build artifacts, logs, and other non-essential files
3. **Verify Output**: Always review the generated PDF to ensure completeness and readability
4. **Maintain Originals**: Keep the original source files as backup to the PDF documentation